

ข้อ 1 การกลับลำดับสตริง (String Reversal)

-คำอธิบายแนวคิด

Recursive Algorithm (reverseRecursive): นำตัวอักษรตัวสุดท้ายของสตริงมาไว้ข้างหน้า แล้วเรียกตัวเองซ้ำกับสตริงส่วนที่เหลือ (ตัดตัวสุดท้ายออก) ไปเรื่อยๆ จนกว่าสตริงจะว่างเปล่า

Iterative Algorithm (reverseIterative): ใช้ลูปวนอ่านตัวอักษรตั้งแต่ตำแหน่งสุดท้ายย้อนกลับมามีตำแหน่งแรก แล้วนำมาต่อกันเป็นสตริงใหม่

-Pseudocode

// Recursive

```
function reverseRecursive(s):
```

```
    if length of s <= 1: return s
```

```
    return last_char(s) + reverseRecursive(s without last_char)
```

// Iterative

```
function reverseIterative(s):
```

```
    result = ""
```

```
    for i from length(s)-1 down to 0:
```

```
        result = result + s[i]
```

```
    return result
```

-โปรแกรมภาษา Java

```
public class StringReverse {
```

```
    // อัลกอริทึมที่ 1: Recursive
```

```
    public static String reverseRecursive(String s) {
```

```
        if (s == null || s.length() <= 1) {
```

```
            return s;
```

```
        }
```

```
        return s.charAt(s.length() - 1) + reverseRecursive(s.substring(0, s.length() - 1));
```

```
    }
```

```
    // อัลกอริทึมที่ 2: Iterative (ใช้ StringBuilder เพื่อประสิทธิภาพ)
```

```
    public static String reverseIterative(String s) {
```

```
        if (s == null) return null;
```

```
        StringBuilder sb = new StringBuilder();
```

```
        for (int i = s.length() - 1; i >= 0; i--) {
```

```
            sb.append(s.charAt(i));
```

```
        }
```

```
        return sb.toString();
```

```
    }
```

```
}
```

-ข้อมูลนำเข้าและผลลัพธ์

Input: "pots&pans"

Output: "snap&stop"

-งานวิเคราะห์

Time Complexity:

Recursive: $\mathcal{O}(n^2)$ หากใช้ + ต่อ String (เพราะ substring และ + สร้าง Object ใหม่ทุกรอบ)
แต่ถ้าพิจารณาเชิงทฤษฎีคือ $\mathcal{O}(n)$

Iterative (ใช้ String +): $\mathcal{O}(n^2)$ / Iterative (ใช้ StringBuilder): $\mathcal{O}(n)$

Space Complexity:

Recursive: $\mathcal{O}(n)$ (จาก Call Stack)

Iterative: $\mathcal{O}(n)$ (สำหรับเก็บข้อความผลลัพธ์)

ผลกระทบของการต่อสตริงด้วย + vs StringBuilder: การใช้ + ในลูปหรือ Recursion ทำให้ต้องสร้าง String Object ใหม่ทุกรอบ ส่งผลให้เสียเวลาและหน่วยความจำ $\mathcal{O}(n^2)$ ส่วน StringBuilder จะแก้ไขข้อมูลใน Buffer เดิมได้ทันที ประสิทธิภาพจึงสูงกว่ามาก

สรุป: Iterative (ร่วมกับ StringBuilder) เหมาะสมกว่า เพราะประหยัด Call Stack และทำงานได้เร็วกว่าเมื่อเจอข้อมูลขนาดใหญ่ ($n = 10,000$)

ข้อ 2 การตรวจสอบ Palindrome

- คำอธิบายแนวคิด

Reverse and Compare (isPalindromeByReverse): สลับข้อความกลับหลังจากหลั้มาหน้า แล้วนำสตริงที่สลับแล้วมาเปรียบเทียบกับสตริงเดิมว่าเหมือนกันหรือไม่

Recursive Two-Pointer (isPalindromeRecursive): ใช้ตัวชี้ตำแหน่งซ้าย (left) และขวา (right) ตรวจสอบตัวอักษรคู่กัน หากเหมือนกันให้เรียกตัวเองซ้ำเพื่อเช็คู่ถัดไปขยับเข้าหาตรงกลาง

-Pseudocode

// Reverse and Compare

function isPalindromeByReverse(s):

 reversed_s = reverse(s)

 return s equals reversed_s

// Recursive Two-Pointer

function isPalindromeRecursive(s, left, right):

 if left >= right: return true

 if s[left] != s[right]: return false

 return isPalindromeRecursive(s, left + 1, right - 1)

-โปรแกรมภาษา Java (รวมเงื่อนไขพิเศษ)

```
public class PalindromeCheck {
```

```
    // ฟังก์ชันช่วยละเว่นตัวพิมพ์เล็ก-ใหญ่ ช่องว่าง และเครื่องหมาย
```

```
    private static String preprocess(String s) {
```

```
        if (s == null) return "";
```

```
        return s.replaceAll("[^a-zA-Z0-9]", "").toLowerCase();
```

```
    }
```

```
    public static boolean isPalindromeByReverse(String s) {
```

```

String clean = preprocess(s);
String reversed = new StringBuilder(clean).reverse().toString();
return clean.equals(reversed);
}

public static boolean isPalindromeRecursive(String s, int left, int right) {
    // กรณีเริ่มต้น ควรทำ preprocess(s) ก่อนส่งเข้าฟังก์ชัน
    if (left >= right) return true;
    if (s.charAt(left) != s.charAt(right)) return false;
    return isPalindromeRecursive(s, left + 1, right - 1);
}
}

```

-ข้อมูลนำเข้าและผลลัพธ์

"racecar" $\rightarrow$ true

"algorithm" $\rightarrow$ false

"A man, a plan, a canal: Panama" $\rightarrow$ true

-งานวิเคราะห์

กรณีี่สุดจริงเป็น Palindrome: สแกนตรวจสอบจนถึงครึ่งทาง

กรณีี่ตัวอักขระคู่แรกไม่ตรงกัน:

Reverse and Compare: ยังคงต้องสลับสตริงทั้งเส้นก่อน ($\mathcal{O}(n)$) แล้วค่อยเช็ค

Recursive Two-Pointer: หยุดทำงานทันทีในการเช็คครั้งแรก (Early Exit)

Best-case Time Complexity:

Reverse & Compare: $\mathcal{O}(n)$

Recursive Two-Pointer: $\mathcal{O}(1)$ (หยุดทันทีเมื่อตัวแรกไม่ตรง)

Worst-case Time Complexity: ทั้งคู่คือ $\mathcal{O}(n)$

Space Complexity:

Reverse & Compare: $\mathcal{O}(n)$

Recursive Two-Pointer: $\mathcal{O}(n)$ (Call Stack)

-สรุป: Recursive Two-Pointer ดีกว่าในประเด็นการประมวลผลจริง เพราะสามารถหยุดทำงานได้ทันทีเมื่อเจอตัวอักขระที่ไม่เข้าคู่กัน (Short-circuit evaluation)

ข้อ 3 การเปรียบเทียบจำนวนสระและพยัญชนะ

-คำอธิบายแนวคิด

Recursive Counting (hasMoreVowelsRecursive): ตรวจสอบตัวอักขระทีละตัวในตำแหน่งปัจจุบันว่าเป็นสระหรือพยัญชนะ จากนั้นบวกผลลัพธ์เข้ากับการเรียกตัวเองซ้ำในตัวอักขระถัดไป

Iterative Counting (hasMoreVowelsIterative): ใช้ลูป for วนอ่านตัวอักขระตั้งแต่ต้นจนจบ พร้อมนับจำนวนสระและพยัญชนะสะสมในตัวแปร

-Pseudocode

```
function hasMoreVowelsIterative(s):  
    vowels = 0, consonants = 0  
    for char in s:  
        if isVowel(char): vowels++  
        else if isConsonant(char): consonants++  
    return vowels > consonants
```

-โปรแกรมภาษา Java

```
public class VowelConsonant {  
    private static boolean isVowel(char c) {  
        c = Character.toLowerCase(c);  
        return c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u';  
    }  
  
    private static boolean isConsonant(char c) {  
        c = Character.toLowerCase(c);  
        return c >= 'a' && c <= 'z' && !isVowel(c);  
    }  
  
    // Iterative  
    public static boolean hasMoreVowelsIterative(String s) {  
        int vowels = 0, consonants = 0;  
        for (int i = 0; i < s.length(); i++) {  
            char c = s.charAt(i);  
            if (isVowel(c)) vowels++;  
            else if (isConsonant(c)) consonants++;  
        }  
        return vowels > consonants;  
    }  
  
    // Recursive Helper  
    public static boolean hasMoreVowelsRecursive(String s) {  
        int[] counts = countRecursive(s, 0);  
        return counts[0] > counts[1]; // index 0: vowels, index 1: consonants  
    }  
  
    private static int[] countRecursive(String s, int index) {  
        if (index >= s.length()) return new int[]{0, 0};  
        int[] res = countRecursive(s, index + 1);  
        char c = s.charAt(index);  
        if (isVowel(c)) res[0]++;  
        else if (isConsonant(c)) res[1]++;  
        return res;  
    }  
}
```

```
}  
}
```

-ข้อมูลนำเข้าและผลลัพธ์

Input: "education"

Vowels: 5, Consonants: 4 $\rightarrow$ Result: true

-วิเคราะห์

Time Complexity: $\mathcal{O}(n)$ ทั้งสองวิธี

Space Complexity:

Iterative: $\mathcal{O}(1)$

Recursive: $\mathcal{O}(n)$ (จาก Call Stack)

Recursive Calls: $n + 1$ ครั้ง

ความเสี่ยงของ StackOverflowError: วิธี Recursive มีความเสี่ยงสูงหากสตริงยาวเกินกว่าความจุของ Call Stack (ประมาณ 10,000+ ตัวอักษรขึ้นไป ขึ้นอยู่กับ JVM)

-ขนาดข้อมูลที่เหมาะสม:

Recursive: สตริงขนาดเล็ก ($n < 1,000$)

Iterative: สตริงขนาดใหญ่ ไม่จำกัดความยาว

ข้อ 4 การจัดกลุ่มจำนวนคู่และจำนวนคี่

-คำอธิบายแนวคิด

Recursive Two-Pointer: ใช้ตัวชี้ left (หาเลขคี่) และ right (หาเลขคู่) สลับค่ากัน แล้วย้ายขอบเขตเรียกฟังก์ชันตัวเองซ้ำ

Iterative Two-Pointer: ใช้ลูปวนซ้ำ left และ right เพื่อสลับตำแหน่งตัวเลขโดยไม่ใช้การเรียกตัวเอง

Extra Array: สร้างอาร์เรย์ใหม่ขนาดเท่าเดิม วนรอบแรกใส่เฉพาะเลขคู่ วนรอบสองใส่เลขคี่

-Pseudocode (Iterative Two-Pointer)

function rearrangeTwoPointer(a):

 left = 0, right = a.length - 1

 while left < right:

 while left < right and a[left] % 2 == 0: left++

 while left < right and a[right] % 2 != 0: right--

 if left < right:

 swap(a[left], a[right])

 left++; right--

-โปรแกรมภาษา Java

```
public class EvenOddPartition {
```

```
    // 1. Recursive Two-Pointer
```

```
    public static void rearrangeRecursive(int[] a, int left, int right) {
```

```
        if (left >= right) return;
```

```

if (a[left] % 2 == 0) {
    rearrangeRecursive(a, left + 1, right);
} else if (a[right] % 2 != 0) {
    rearrangeRecursive(a, left, right - 1);
} else {
    int temp = a[left];
    a[left] = a[right];
    a[right] = temp;
    rearrangeRecursive(a, left + 1, right - 1);
}
}

```

// 2. Iterative Two-Pointer

```

public static void rearrangeTwoPointer(int[] a) {
    int left = 0, right = a.length - 1;
    while (left < right) {
        while (left < right && a[left] % 2 == 0) left++;
        while (left < right && a[right] % 2 != 0) right--;
        if (left < right) {
            int temp = a[left];
            a[left] = a[right];
            a[right] = temp;
            left++;
            right--;
        }
    }
}

```

// 3. Extra Array (Stable Version)

```

public static int[] rearrangeExtraArray(int[] a) {
    int[] result = new int[a.length];
    int idx = 0;
    for (int val : a) {
        if (val % 2 == 0) result[idx++] = val;
    }
    for (int val : a) {
        if (val % 2 != 0) result[idx++] = val;
    }
    return result;
}
}

```

-วิเคราะห์และเปรียบเทียบ
คุณสมบัติ.

Recursive Two-Pointer.

Time Complexity.	$\{O\}(n)$
Space Complexity.	$\{O\}(n)$
จำนวนครั้งการสลับ.	$n/2$ ครั้ง
การเปลี่ยนอาร์เรย์เดิม.	เปลี่ยน (In-place)
Stable Algorithm.	ไม่ Stable

คุณสมบัติ.	Iterative Two-Pointer
Time Complexity.	$\{O\}(n)$
Space Complexity.	$\{O\}(1)$
จำนวนครั้งการสลับ.	$n/2$ ครั้ง
การเปลี่ยนอาร์เรย์เดิม.	เปลี่ยน (In-place)
Stable Algorithm.	ไม่ Stable

คุณสมบัติ.	Extra Array
Time Complexity.	$\{O\}(n)$
Space Complexity.	$\{O\}(1)$
จำนวนครั้งการสลับ.	0 ครั้ง (คัดลอกลงอาร์เรย์ใหม่)
การเปลี่ยนอาร์เรย์เดิม.	ไม่เปลี่ยน (สร้างใหม่)
Stable Algorithm.	Stable (รักษาลำดับเดิม)

หมายเหตุเรื่อง Stability: วิธี Two-Pointer สลับข้อมูลข้ามตำแหน่งทำให้ลำดับเดิมของตัวเลขเสียไป ในขณะที่วิธี Extra Array สามารถรักษาสภาพ Stable ได้ (เช่น Input [5, 2, 7, 4, 9, 6] ได้ Stable Output [2, 4, 6, 5, 7, 9])

ข้อ 5 การแบ่งอาร์เรย์ตามค่า k

-คำอธิบายแนวคิด

Recursive Partition: ใช้ตัวชี้ซ้าย/ขวา ค้นหาตัวเลขที่ $> k$ ผังซ้าย และ $\leq k$ ผังขวา แล้วสลับกันแบบ Recursive

Iterative Partition: ใช้ลูปวนสลับข้อมูลในลักษณะเดียวกับขั้นตอน Partition ของ Quick Sort

Sorting-Based: เรียงลำดับข้อมูลทั้งอาร์เรย์จากน้อยไปมาก เพื่อให้ตัวเลขที่ $\leq k$ ไปอยู่ด้านหน้าทั้งหมด

-โปรแกรมภาษา Java

```
import java.util.Arrays;
```

```
public class PartitionByK {
    // 1. Recursive Partition
    public static void partitionRecursive(int[] a, int k, int left, int right) {
        if (left >= right) return;
        if (a[left] <= k) partitionRecursive(a, k, left + 1, right);
        else if (a[right] > k) partitionRecursive(a, k, left, right - 1);
        else {
            int temp = a[left];
            a[left] = a[right];
            a[right] = temp;
        }
    }
}
```

```

        partitionRecursive(a, k, left + 1, right - 1);
    }
}

// 2. Iterative Partition
public static void partitionIterative(int[] a, int k) {
    int i = 0;
    for (int j = 0; j < a.length; j++) {
        if (a[j] <= k) {
            int temp = a[i];
            a[i] = a[j];
            a[j] = temp;
            i++;
        }
    }
}

// 3. Sorting-Based
public static void partitionBySorting(int[] a, int k) {
    Arrays.sort(a);
}
}

```

-งานวิเคราะห์

วิธี.	Time Complexity
Recursive Partition.	$\{O\}(n)$
Iterative Partition.	$\{O\}(n)$
Sorting-Based.	$O(n \log n)$

วิธี.	Space Complexity
Recursive Partition.	$\{O\}(n)$
Iterative Partition.	$\{O\}(1)$
Sorting-Based.	$\{O\}(1)$ หรือ $\{O\}(\log n)$

วิธี.	In-place?
Recursive Partition.	ใช่ (In-place ในแง่อาร์เรย์ แต่ใช่ Stack)
Iterative Partition.	ใช่ (In-place แท้จริง)
Sorting-Based.	ใช่

เหตุผลที่ Sorting ทำงานช้ากว่าที่จำเป็น: ปัญหาต้องการเพียงแค่ "แยกกลุ่ม" $\leq k$ และ $> k$ (ใช้เวลา $\mathcal{O}(n)$) แต่การ Sorting ทำงานเกินจำเป็นโดยไปเรียงลำดับสมาชิกทุกตัวภายในกลุ่มด้วย ทำให้เสียเวลาเพิ่มขึ้นเป็น $\mathcal{O}(n \log n)$

ความสัมพันธ์กับ Quick Sort: ปัญหานี้คือขั้นตอน Lomuto/Hoare Partition Scheme ซึ่งเป็นหัวใจหลักในการเลือก Pivot เพื่อแบ่งข้อมูลของอัลกอริทึม Quick Sort

วิธีที่ทำ In-place ได้: Iterative Partition และ Recursive Partition

ข้อ 6 การค้นหาจำนวนที่มีผลรวมเท่ากับ k

-คำอธิบายแนวคิด

Brute Force: ตรวจสอบสมาชิกทุกคู่ที่เป็นไปได้ด้วย Nested Loop ($N \times N$)

Recursive Two-Pointer: เนื่องจากอาร์เรย์เรียงลำดับแล้ว ใช้ตัวชี้ left (เริ่มต้น) และ right (สิ้นสุด) ถ้าผลรวมน้อยกว่า k ให้ขยับ left ขึ้น ถ้ามากกว่า k ให้ลด right ลง

Binary Search: วนลูปเลือกสมาชิกทีละตัว $A[i]$ แล้วใช้วิธี Binary Search หาค่าคู่สม $k - A[i]$ ในช่วงอาร์เรย์ที่เหลือ

-โปรแกรมภาษา Java

```
public class FindPairSum {  
    // 1. Brute Force  
    public static boolean findPairBruteForce(int[] a, int k) {  
        for (int i = 0; i < a.length; i++) {  
            for (int j = i + 1; j < a.length; j++) {  
                if (a[i] + a[j] == k) return true;  
            }  
        }  
        return false;  
    }  
  
    // 2. Recursive Two-Pointer  
    public static boolean findPairRecursive(int[] a, int k, int left, int right) {  
        if (left >= right) return false;  
        int sum = a[left] + a[right];  
        if (sum == k) return true;  
        if (sum < k) return findPairRecursive(a, k, left + 1, right);  
        return findPairRecursive(a, k, left, right - 1);  
    }  
  
    // 3. Binary Search  
    public static boolean findPairBinarySearch(int[] a, int k) {  
        for (int i = 0; i < a.length; i++) {  
            int complement = k - a[i];  
            int low = i + 1, high = a.length - 1;  
            while (low <= high) {  
                int mid = low + (high - low) / 2;  
                if (a[mid] == complement) return true;  
                if (a[mid] < complement) low = mid + 1;  
                else high = mid - 1;  
            }  
        }  
        return false;  
    }  
}
```

```
}  
}
```

เปรียบเทียบเชิงวิเคราะห์

อัลกอริทึม.	แนวคิด
Brute Force.	ตรวจสอบทุกคู่
Recursive Two-Pointer.	ลดขอบเขตการค้นหาทีละตำแหน่ง
Binary Search.	ค้นหาค่าคู่สมของสมาชิกแต่ละตัว

อัลกอริทึม.	Time Complexity
Brute Force.	$O(n^2)$
Recursive Two-Pointer.	$O(n)$
Binary Search.	$O(n \log n)$

อัลกอริทึม.	Space Complexity
Brute Force.	$O(1)$
Recursive Two-Pointer.	$O(n)$
Binary Search	$O(1)$

เพราะเหตุใด Two-Pointer จึงใช้ได้เมื่ออาร์เรย์เรียงลำดับแล้ว?

ตอบ: เพราะทิศทางการปรับตัวที่มีความแน่นอน เช่น ถ้า $A[\text{left}] + A[\text{right}] < k$ เรามั่นใจได้ว่าการขยับ left ไปทางขวาจะทำให้ผลรวมเพิ่มขึ้นอย่างแน่นอน (เพราะข้อมูลเรียงจากน้อยไปมาก)

จะเกิดอะไรขึ้นหากนำไปใช้กับอาร์เรย์ที่ยังไม่เรียงลำดับ?

ตอบ: จะเกิด (Incorrect Result) เพราะไม่สามารถคาดเดาได้ว่าการขยับ left หรือ right จะทำให้ผลรวมเพิ่มขึ้นหรือลดลง ส่งผลให้อัลกอริทึมข้ามคู่คำตอบที่ถูกต้องไป

ตอบคำถามท้ายการทดลอง

1 ผลการทดลองสอดคล้องกับ Big-O ที่วิเคราะห์ไว้หรือไม่?

ตอบ: สอดคล้อง: อัลกอริทึมที่มี Time Complexity สูงกว่า เช่น $\mathcal{O}(n^2)$ จะใช้เวลาเติบโตแบบก้าวกระโดดเมื่อ n เพิ่มขึ้น ในขณะที่ $\mathcal{O}(n)$ เวลาจะเพิ่มขึ้นเป็นสัดส่วนเส้นตรง

2 อัลกอริทึมใดเร็วที่สุดเมื่อข้อมูลมีขนาดเล็ก?

ตอบ: เมื่อข้อมูลขนาดเล็ก ($n = 100$) ความแตกต่างจะน้อยมาก บางครั้งอัลกอริทึมแบบ Simple/Iterative หรือ Brute Force อาจเร็วที่สุด เนื่องจากไม่มี Overhead ของการจัดการ Memory/Call Stack

3 อัลกอริทึมใดเหมาะสมที่สุดเมื่อข้อมูลมีขนาดใหญ่?

ตอบ: อัลกอริทึมที่มี Big-O ต่ำที่สุดและเป็น Iterative (เช่น $\mathcal{O}(n)$ Iterative Two-Pointer) เหมาะสม

4 Recursive Algorithm มีข้อจำกัดด้านหน่วยความจำอย่างไร?

ตอบ: ใช้หน่วยความจำประเภท Call Stack เพิ่มขึ้นตามจำนวนชั้นการเรียกตัวเอง ($\mathcal{O}(n)$ Space) หากข้อมูลมีขนาดใหญ่มากจะทำให้เกิด StackOverflowError

5 เหตุใดอัลกอริทึมที่มี Big-O เท่ากันจึงอาจใช้เวลาจริงต่างกัน?

ตอบ: เพราะ Big-O ตัดค่าคงที่ (Constant Factors) และ Constant Overhead ออก เช่น $\mathcal{O}(2n)$ และ $\mathcal{O}(100n)$ มี Big-O เท่ากับ $\mathcal{O}(n)$ เหมือนกัน แต่การทำงานจริงมีจำนวนคำสั่งประมวลผล CPU ต่างกัน 50 เท่า

6 การวัดเวลาเพียงครั้งเดียวเพียงพอหรือไม่ เพราะเหตุใด?

ตอบ: ไม่เพียงพอ: เพราะเวลาที่ได้ในการรันครั้งเดียวอาจมีความคลาดเคลื่อน (Noise) จากสถานะของระบบ เช่น การทำงานของ Background Process อื่นๆ หรือระบบ Garbage Collection (GC) ของ Java จึงจำเป็นต้องรันหลายๆ ครั้งแล้วหาค่าเฉลี่ย